Network Traffic Analysis for DFIR: Forensic Report

Nov 2, 2026 · @Tribbiani

Summary

I studied how network traffic analysis works in DFIR (digital forensics and incident response), with two goals: to learn the method and to build a behavioral baseline of my own network. I made 16 captures between October 25 and 31, 2026, using Wireshark on the Wi-Fi adapter (no monitor mode) on Windows 11 with WSL2, and analyzed them with tshark, Nmap, the Opera GX browser, and Python scripts. Everything was done on my own network and my own computer. Port scans were run only against my phone and my TV, with the family members who live with me informed and in agreement. Capture 16 was made out of personal curiosity and deleted after analysis: its content is not described, and only aggregate results from it appear here, used as a control.

Three findings are supported with high certainty. First, QUIC traffic (a transport protocol over UDP) to dns.google exists only while Opera GX is running: seven captures were compared, changing one condition at a time, and captures 06 and 07 differ only in the Opera processes; the likely purpose is DNS over HTTPS (medium certainty). Second, when the browser is started with a TLS key log file (the SSLKEYLOGFILE variable), Wireshark reads the browser's own traffic in clear text: GET lines, 200 OK, headers, user-agent, and server. This works only on my own computer and does not intercept or alter the connection, unlike a proxy. Without the keys, the certificate and content of TLS 1.3 stay encrypted, and only the SNI (the server name, sent in clear text), IPs, ports, timings, and sizes remain visible. Third, a port scan is recognized by counting: 1,000 distinct ports in 1.17 s from the same source. A sliding-window script detected it, and normal browsing (capture 16) did not trigger the alert. With medium certainty, the JA4 fingerprint shows the software family (Opera's is the same in captures 08 and 16, and Opera and Spotify share the cipher hash because both use the Chromium engine), but not the exact application.

Open items: device .34, which sent hundreds of unsolicited ARP replies (141 in 35 s in capture 09 and 453 in capture 10); a few unowned devices that I would have to check one by one with their owners and that did not affect the findings above; who talks to Microsoft telemetry; and the real service behind each open port on the TV, which Nmap only guessed from the port number (version detection, -sV, would settle it). The Open

cases section lists the other limits. With the baseline, port-scan detection, and the detection window in hand, the next step is an incident analysis: a harmless, local-only script that imitates malware behavior, so the network can be compared with the baseline.

Scope, authorization, and limits

The study covers only my home network and my own devices, and this report contains no third-party traffic.

- **What was captured:** my computer's traffic, plus what the network announces to everyone (ARP, mDNS, SSDP). With the Wi-Fi adapter in normal mode, the computer does not receive packets addressed to other devices.
- **Scans:** device discovery (ARP) queried every address in the subnet. Port scans were run against only two devices of mine: my iPhone and the TV in my bedroom.
- **Authorization:** the network belongs to my family, and everyone was informed and agreed to the analysis.
- **One study capture (16)** was deleted after analysis, and its content is not described in this report.
- **What is not published:** captures (pcap), TLS key files, and derived CSVs stay on my computer only. The scripts and this report may be published, as they have already been sanitized.

Environment and methodology

All captures were made with Wireshark on my own computer, on my home network, following a capture sheet and checking every file by hash.

- **Environment:** Windows 11 with WSL2 (Ubuntu), Wireshark (capturing on the Wi-Fi adapter), tshark, Nmap 7.96, Python (Miniconda), and VS Code. The network is 192.168.0.0/24, with the router at 192.168.0.1.
- **Capture sheet:** before each capture I record the open applications (checked in Task Manager, Details tab), the goal, and the time. The lesson came from capture 06, where Opera GX looked closed but had two background processes.
- **One variable at a time:** to attribute traffic to a program, I change one condition and repeat the capture (the dns.google case).
- **Integrity:** the SHA-256 of each capture is computed with `Get-FileHash` and recorded in the log.
- **Extraction and analysis:** tshark exports only the relevant fields to CSV, and Python scripts count and compare them (appendix).

- **Decrypting my own traffic:** the browser is started with the `SSLKEYLOGFILE` variable and Wireshark reads the key file. This works only for my own computer's traffic, and the key file never leaves it.

How I state certainty

- **High:** two independent pieces of evidence, or checked on the device itself.
- **Medium:** one strong piece of evidence, but with a possible alternative explanation.
- **Low:** a hypothesis not yet tested.

I write "lookup of domain X", not "visited site X", because the name in the SNI proves neither which program made the request nor the intent of the person at the device.

Baseline

On an ordinary day the network has about 12 devices visible and my PC talks to a few fixed services; every anomaly was measured against this.

What is normal	Evidence	Certainty
Idle network: ARP, NDP and MLD (IPv6), SSDP, and mDNS from devices announcing themselves	Capture 01 (136 packets)	high
Ordinary DNS for most sites, using 8.8.8.8 and the ISP resolver	Captures 02 to 04	high
Microsoft telemetry (mobile.events.data.microsoft.com), with a Windows-type client (JA4 t13d201200)	Captures 03, 05 and 07	medium
QUIC traffic to dns.google, only while Opera GX is running	Captures 01 to 07, with and without Opera	high
Spotify: TCP and QUIC connections to Spotify's own domains	Captures 05, 07 and 08	high
Opera GX: the browser's own services (updates, extensions, start page) right after launch	Captures 08 and 16	high
Applications usually open: Opera GX, Spotify, Claude Code, VS Code, and the terminal	My routine; only partly verified	medium

Baseline points still unexplained: an 802.1Q (VLAN) line that accounted for 23.5% of the frames in capture 01, and UDP packets to port 15600 (looks like BSDP, not confirmed).

Capture timeline

There were 16 captures between October 25 and 31, 2026, each with one question and a known condition; capture 16 was deleted and is not detailed. The dates of captures 02 to 07 are approximate.

No.	When	Goal and conditions	Result
01	Oct 25	Idle network	136 packets; ARP, NDP, MLD, SSDP and mDNS
02	Oct 25 to 27	Open a website, focus on DNS	23,898 packets, 1,116 of them DNS; the browser asks for HTTPS (type 65), AAAA and A at the same time
03	Oct 25 to 27	nslookup against 8.8.8.8 and the ISP resolver, browser open	7,626 packets; nslookup makes extra queries (PTR and the .lan suffix); the TTL of the same record varies
04	Oct 25 to 27	Race between two resolvers, 4 rounds, browser closed	490 packets; no winner with a single sample
05	Oct 25 to 27	Open a website in Opera GX, focus on the Client Hello	373 packets; certificate invisible (TLS 1.3); QUIC to dns.google
06	Oct 27	Spotify only (but Opera GX had 2 hidden processes)	Contaminated capture; lesson in method
07	Oct 27	Spotify only, processes verified	1,550 packets; no traffic to dns.google
08	Oct 27	Visit with the TLS key file active	Decrypted traffic (HTTP/1.1, 2 and 3); server behind Cloudflare
09	Oct 29, 10:40	Nmap discovery (ARP), default	Nmap listed 4 devices; the capture showed late replies from others
10	Oct 29, 11:01	Nmap discovery, patient mode (-T2)	6 devices in 305 s
11	Oct 29, 11:35	3 ports on the iPhone	2 closed, 1 open (62078)

No.	When	Goal and conditions	Result
12	Oct 29, 13:35	Ports 1 to 1000 on the iPhone	1,000 SYN in 1.17 s and 1,000 RST; none open
13	Oct 31, 16:17	Ports 1 to 1000 on the bedroom TV	Same as the iPhone: none open
14	Oct 31, 16:25	All ports on the TV	14 open ports
15	Oct 31	VS Code only	Microsoft telemetry appeared with 3 different JA4 fingerprints
16	Oct 31, 18:22	Study of a visit (deleted)	140 Client Hellos; Opera fingerprints reproduced; the scan alert did not fire

Findings

Eight findings are supported by the captures; the three strongest are dns.google attributed to Opera GX, decryption of my own traffic, and port-scan detection by counting ports.

Finding	Evidence	Certainty	Limits
QUIC traffic to dns.google exists only while Opera GX is running	Seven captures compared, changing one condition at a time; captures 06 and 07 differ only in the Opera processes	high	The purpose (probably DNS over HTTPS) is medium; Opera's DoH option is off and the mechanism is unknown; the conditions of captures 02 and 03 were noted from memory, and the strongest proof is the 06/07 pair
With the key file, the browser's own traffic is read in clear text	Capture 08: GET lines, 200 OK and HEADERS; user-agent OPR/137; server: cloudflare	high	Applies only to my computer and requires the key file; the user-agent is a label that can be forged

Finding	Evidence	Certainty	Limits
In TLS 1.3 the certificate and content are encrypted; the SNI, IPs, ports, timings, and sizes remain visible	Captures 05 and 16: 0 Certificate messages; SNI in clear text	high	The SNI shows the destination, not the program or what was done on the site
The iPhone's Wi-Fi address changes: .107 (Oct 25) and .17 (Oct 29)	Address checked in the phone's settings	high for .17; medium that they are the same device	Rotating private address; the MAC is not a stable identifier, so each record carries its date
Active discovery finds more than listening, but not everything	Captures 09 and 10: ARP replies arrived 5 to 15 s later, after Nmap had given up; patient mode found 6, not 4	medium	Power-saving as the explanation is a hypothesis; devices come and go on the network
Open ports: the iPhone has 1 (62078) and the bedroom TV has 14, none below 1000	Captures 11 to 14, with SYN/ACK on open ports and RST on closed ones	high	The service name comes from a table by port number and is only a guess (low); "open" does not mean dangerous
A 1,000-port scan is recognized by counting: 1,000 distinct ports in 1.17 s, from the same source and with the same source port	Captures 12 and 13; sliding-window script; negative control in capture 16 (normal browsing uses 1 port per destination)	high	A slow scan escapes a short window; a scan is reconnaissance, not proof of intrusion
The same program keeps its JA4 fingerprint across captures, but JA4 shows only the software family	Opera's JA4 is the same in captures 08 and 16; Opera and Spotify share the cipher hash (Chromium engine)	medium	Can be imitated; does not separate programs with the same engine

Open cases and gaps

Seven questions remain unanswered, and each has a clear way to be closed.

Case	What is known	How to close it
Who talks to Microsoft telemetry	At least 4 client families reached the same server (captures 03 to 15). VS Code did not reproduce the fingerprint from capture 08; Spotify is a weak candidate	Capture one program at a time, restarting the program inside the capture and checking the processes
Why device .34 sends hundreds of ARP replies with no one asking	141 in 35 s in capture 09 and 453 in capture 10; content consistent with its own MAC	Identify the device (ipconfig /all) and capture with it turned off
Unowned devices: .74 (Samsung), .170 (Apple), .222 and .242 (random MAC)	Answered in some scans or appear only in arp -a	Check the Wi-Fi address on the devices and the router's list, with authorization
My PC's connection to the other TV (.152) on ports 7678 and 8080	A normal connection, with data, that did not come from Nmap	Follow the TCP stream and find the source program
Mechanism of the traffic to dns.google in Opera GX	Depends on Opera running; the DoH option is off	Capture with keys and compare Opera's startup with and without components
The real service behind each open port on the TV	Nmap only guessed from the number	Run version detection (-sV) on my own devices only
The 802.1Q line (23.5% of capture 01) and UDP on port 15600	Seen in the idle capture	Open the packets and read the headers

Conclusions and lessons in method

- **A capture proves facts about traffic, not intent or illegal acts.** That is why I write "lookup of domain X", not "visited site X": the name in the SNI proves neither which program made the request nor what the person did there.

- **One variable at a time, and a capture sheet.** Capture 06 appeared to have only Spotify, but Opera GX had two hidden processes; only capture 07, with the processes verified, allowed a conclusion.
- **A condition noted from memory is not a verified condition.** The conditions of captures 02 and 03 were noted without checking, so the strongest proof about dns.google is the 06/07 pair.
- **Baseline and negative control before calling anything an anomaly.** The scan alert has value only because normal browsing (about 1 port per destination) does not trigger it.
- **Encryption limits what is visible, and a key changes that only on your own computer.** In TLS 1.3 the certificate and content are encrypted; the SNI, IPs, ports, timings, and sizes remain visible and already say a lot.
- **A rotating MAC address is not a stable identifier.** The iPhone showed different addresses on different days; every device record needs a date.
- **Detecting is not stopping, and a scan is not an intrusion.** Port counting recognizes reconnaissance; a slow scan escapes a short window.
- **In a real case, traffic is one piece of the evidence.** It would be cross-checked against host artifacts (registry, browser history, USB devices), with a chain of custody and a hash of each capture. Capturing other people's traffic requires authorization.

Appendices

The scripts and commands let the analysis be repeated on any new capture. Script names and options are in Portuguese.

A. Python scripts

Script	What it does
contar_portas.py	Version 1, didactic: counts distinct ports per source/destination pair over the whole capture, with no time window
contar_portas_v3.py	Looks for scans: counts distinct ports per source and destination in a sliding window (--limite = threshold, --janela = window)
portas_abertas.py	Lists open ports (SYN/ACK) and how many times each one answered
contar_ja4.py	Groups Client Hellos by JA4 fingerprint and lists the names (SNI) of each group

Script	What it does
comparar_ja4.py	Compares the fingerprints of one name across several captures

The scripts read CSVs exported by tshark, with filters such as `tcp.flags.syn==1 and tcp.flags.ack==0` (scan) and `tls.handshake.type==1` (Client Hello). The full commands are in the `comandos.md` guide. Limitation: the SYN and SYN/ACK scripts look at IPv4 only.

B. Glossary

- **SNI:** the server name, sent in clear text at the start of a TLS connection.
- **JA3 and JA4:** a fingerprint of how a program speaks TLS, computed from the Client Hello.
- **SYN, SYN/ACK and RST:** the packets of the TCP handshake; an open port answers SYN/ACK and a closed port answers RST.
- **QUIC:** a transport over UDP that carries HTTP/3 and already includes TLS 1.3.
- **TTL:** in IP, the hop limit; in DNS, the remaining validity of a cached answer.
- **DoH:** DNS over HTTPS, where DNS queries travel inside encrypted web traffic.